

DRY-RUN GUIDE | app.js
Source: ca-2\fewd\src\app.js

CORE BEHAVIOR

Defines ten pure task/score utilities. The file contains no invocation or export in this excerpt, so loading it alone prints nothing; the dry run demonstrates representative calls to the functions.

REPRESENTATIVE INPUT / CALL

Sample calls: `greet('Aisha')`; `isPassing(40)`; `sumScores([70,80])`;
`highScores([10,50,90],40)`; `formatTask({title:'Ship',done:true,priority:'low'})`.

STEP TRACE

`greet` builds a template string; boundary 40 passes; reduce totals 150; filter keeps 50 and 90; `formatTask` selects `[x]` and appends title/priority.

EXPECTED OUTPUT / RETURN

Returns: `'Hi, Aisha! Welcome back.'`; `true`; `150`; `[50,90]`; `'[x] Ship (low)'`. No output when file runs by itself.

ACTUAL SOURCE EXCERPT

Verbatim lines from the source; long lines wrap. Source files were not modified.

```
001 | /*
002 |  * Module 2 Practice Assessment - Task Tracker data utilities
003 |  * -----
004 |  * Implement the 10 functions below. Each one is checked by a Jasmine
005 |  * spec in tests/FunctionsTest.js (10 specs x 2 marks = 20).
006 |  *
007 |  * A "task" is a plain object shaped like:
008 |  *   { title: "Buy milk", status: "todo", done: false, priority: "high" }
009 |  *
010 |  * Rules:
011 |  *   - Do NOT rename the functions and do NOT change their parameters.
012 |  *   - Do NOT edit index.html, main.css, or tests/FunctionsTest.js.
013 |  *   - Keep each function pure: return a value, do not print or mutate
014 |  *     the inputs unless the task explicitly says to.
015 |  * -----
016 | */
017 |
018 | // 1. greet(name) -> a greeting built with a TEMPLATE LITERAL.
019 | //   greet("Aisha") must return exactly: "Hi, Aisha! Welcome back."
020 | function greet(name) {
021 |   return `Hi, ${name}! Welcome back.`;
022 | }
023 |
024 | // 2. isPassing(score) -> true if score is 40 or more, else false.
025 | function isPassing(score) {
026 |   return score >= 40;
027 | }
028 |
029 | // 3. sumScores(scores) -> the sum of all numbers in the array.
030 | //   sumScores([]) must return 0.
031 | function sumScores(scores) {
032 |   return scores.reduce((total, score) => total + score, 0);
033 | }
```

```

034 |
035 | // 4. highScores(scores, min) -> a NEW array with only the scores
036 | //   strictly greater than min. highScores([10,50,90], 40) -> [50,90]
037 | function highScores(scores, min) {
038 |   return scores.filter(score => score > min);
039 | }
040 |
041 | // 5. titles(tasks) -> an array of every task's .title, in order.
042 | function titles(tasks) {
043 |   return tasks.map(task => task.title);
044 | }
045 |
046 | // 6. countByStatus(tasks, status) -> how many tasks have that .status.
047 | function countByStatus(tasks, status) {
048 |   return tasks.filter(task => task.status === status).length;
049 | }
050 |
051 | // 7. toggleDone(task) -> a NEW task object identical to the input but
052 | //   with .done flipped. The ORIGINAL object must not be changed.
053 | function toggleDone(task) {
054 |   return { ...task, done: !task.done };
055 | }
056 |
057 | // 8. longestTitle(tasks) -> the single .title string with the most
058 | //   characters. Assume at least one task; ties may return either.
059 | function longestTitle(tasks) {
060 |   return tasks.reduce((longest, task) =>
061 |     task.title.length > longest.length ? task.title : longest
062 |   );
063 | }
064 |
065 | // 9. averageScore(scores) -> the average of the numbers.
066 | //   averageScore([]) must return 0 (do not divide by zero).
067 | function averageScore(scores) {
068 |   if (scores.length === 0) {
069 |     return 0;
070 |   }
071 |
072 |   return sumScores(scores) / scores.length;
073 | }
074 |
075 | // 10. formatTask(task) -> a display line built with a TEMPLATE LITERAL:
076 | //   done tasks start with "[x] ", not-done with "[ ] ", then the
077 | //   title, then " (priority)".
078 | //   { title:"Ship", done:true, priority:"low" } -> "[x] Ship (low)"
079 | function formatTask(task) {
080 |   const marker = task.done ? "[x]" : "[ ]";
081 |   return `${marker} ${task.title} (${task.priority})`;
082 | }
083 |

```